PRISM: DDPM, DDIM y *Classifier-Free Guidance* sobre MNIST y CIFAR-10 con una sola GPU

Josu Viteri Gotzon Viteri

https://github.com/josuviteri/PRISM

Resumen

PRISM es un proyecto de código abierto que implementa los modelos fundacionales DDPM, DDIM y *classifier-free guidance* (CFG) para MNIST y CIFAR-10 en PyTorch. Este documento describe el código del repositorio: el proceso de difusión, la U-Net condicional, el entrenamiento con EMA, el muestreo, el cálculo de FID e IS, una aplicación web y los tests. Los modelos se entrenaron en una RTX 5070 de 12 GB durante 38,6k a 70,3k pasos. La evaluación actualizada usa etiquetas equilibradas en los modelos condicionales y permite variar la escala CFG. En CIFAR-10, el mejor resultado medido en el barrido realizado es FID 36,04 e IS 8,28 con $w = 2$. Los resultados de MNIST siguen bajo investigación por el comportamiento del evaluador LeNet.

1. Introducción

El repositorio reúne 83 *commits* de ambos autores (Josu Viteri y Gotzon Viteri) y usa licencia Apache-2.0. Implementa DDPM [1], DDIM [2] y CFG [3], y una interfaz web que muestra el ruido inicial, la imagen final y varios estados intermedios del muestreo. Es un trabajo de asignatura con una regla 5/20 (un *smoke test* de menos de 5 min en CPU) y un comprobador de reproducibilidad propio. Las métricas de este documento son las que figuran en el README; no se han recalculado.

2. Formulación implementada

Proceso directo. Con $T = 1000$ y β_t lineal entre 10^{-4} y 0,02 (calculado en `float64`), $x_t = \sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon$, con imágenes en $[-1, 1]$. La pérdida es $\mathcal{L} = \mathbb{E}\|\epsilon - \epsilon_\theta(x_t, t, c)\|^2$ con t uniforme en $\{0, \dots, T-1\}$ y *autocast* bfloat16 en GPU.

CFG. Con probabilidad `p_uncond` la etiqueta se cambia por una clase nula (índice $K = 10$). En muestreo,

$$\tilde\epsilon = \epsilon_\theta(x_t, \varnothing) + w\,[\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \varnothing)]. \qquad (1)$$

Con $w = 1$ el código devuelve la predicción condicional y evita la segunda pasada. La escala puede sobrescribirse en evaluación mediante `-guidance-scale`. Esta operación solo es válida si el *checkpoint* se entrenó con $p_{\text{uncond}} > 0$.

Muestreo. En cada paso se calcula $\hat x_0 = (x_t - \sqrt{1-\bar\alpha_t}\,\tilde\epsilon)/\sqrt{\bar\alpha_t}$ y se recorta a $[-1, 1]$. DDPM usa la media posterior $\tilde\mu(x_t, \hat x_0)$ y varianza β_t en los T pasos. DDIM ($\eta = 0$) recorre S pasos equiespaciados (50 por defecto) con $x_s = \sqrt{\bar\alpha_s}\hat x_0 + \sqrt{1-\bar\alpha_s}\,\tilde\epsilon$. Una sola función implementa ambos métodos y devuelve hasta 16 fotogramas de la trayectoria para la web.

3. Implementación

Organización. `run.py` (166 líneas) solo orquesta. Los hiperparámetros están en `configs/` (YAML). El resto se reparte en `src/data` (carga y *splits*), `src/models` (U-Net y LeNet), `src/utils` (difusión, W&B, semilla), `src/training`, `src/app` (servidor), `scripts/` (descarga, inferencia y métricas) y `tests/` (26 funciones). En total hay unas 4,6k líneas de Python.

U-Net. Bloques residuales con GroupNorm y SiLU, embedding sinusoidal del tiempo proyectado a 4·ch y sumado en cada bloque, atención de una cabeza (`scaled_dot_product_attention`) en las resoluciones configuradas, conexiones de salto y *upsampling* por interpolación seguido de convolución. La segunda convolución de cada bloque residual, la proyección de la atención y la capa de salida parten de pesos nulos. La clase se codifica con un `Embedding`$(K+1)$ que se suma al embedding temporal. La Figura 2 resume la estructura.

	CIFAR-10	**MNIST**
Canales base / multiplicadores	128 / [1,2,2,2]	64 / [1,2,4]
Bloques res. / atención en	2 / 16	2 / 14, 7
Dropout / tamaño de lote	0,1 / 128	0,1 / 128
Aumento de datos	volteo horiz.	ninguno
Épocas sin guía / con guía	130 / 150	110 / 200
`p_uncond`, w (con guía)	0,1, 3,0	0,1, 3,0

Tabla 1: Configuraciones. Comunes: Adam, lr $= 2 \cdot 10^{-4}$, *warm-up* de 1000 pasos, recorte de gradiente 1,0, EMA 0,9995 con *warm-up*, `torch.compile`, semilla 42.

Entrenamiento. Adam con *warm-up* lineal y recorte de gradiente (se registra la norma previa al recorte). La EMA usa decaimiento efectivo mín(decay, $\frac{1+n}{10+n}$). El bucle comprueba valores no finitos, guarda *checkpoints* de forma atómica junto con el estado de reanudación y genera una rejilla de muestras EMA cada 10 épocas. Todo se registra en W&B. La pérdida de validación usa los pesos EMA y una semilla fija.

Datos. El *split* entrenamiento/validación (10 %) usa semilla y se hace antes de calcular estadísticas. La normalización de MNIST se calcula solo con entrenamiento y el aumento solo se aplica a entrenamiento. `get_data.py` verifica SHA-256 y el lector IDX de MNIST valida las cabeceras.

Barrido de escala CFG. Para CIFAR-10 se evaluó el *checkpoint* guiado con DDIM de 50 pasos y 10 000 muestras, usando etiquetas equilibradas:

En CIFAR-10 el IS aumenta con w, mientras que el FID presenta un mínimo en $w = 2$. Es el mejor valor de los probados, no necesariamente el óptimo global. El modelo incondicional produjo FID 41,2357 e IS 6,9424

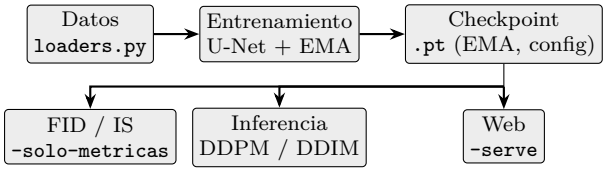


Figura 1: Flujo de trabajo: los tres usos del *checkpoint* leen los pesos EMA.

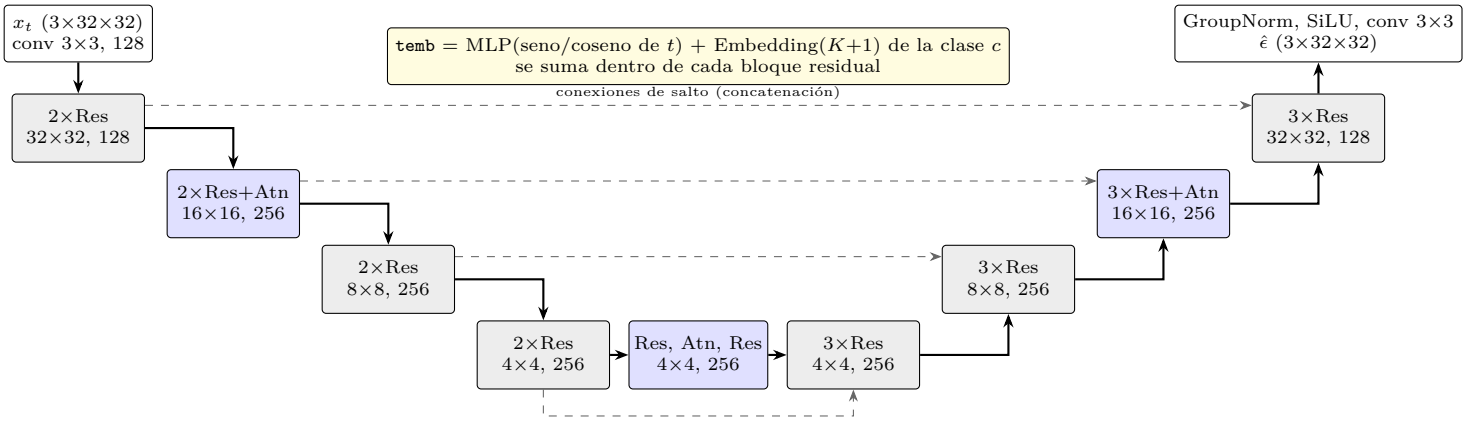


Figura 2: U-Net con la configuración de CIFAR-10 (128 canales base, multiplicadores [1,2,2,2]). Res: bloque residual (GroupNorm, SiLU, `temb`), Atn: atención de una cabeza. El descenso usa convoluciones 3×3 de paso 2 y el ascenso interpolación del vecino más cercano con convolución 3×3; el ascenso tiene un bloque más por nivel porque concatena los mapas de salto. La red de MNIST tiene tres niveles (28, 14 y 7 píxeles; 64, 128 y 256 canales; atención a 14 y 7).

Modelo	Pasos	Conjunto de test	FID↓	IS↑
DDPM original, CIFAR-10 [1]	800k	completo	3,17	9,46
MNIST				
`mnist_ddpm_v0.2`	38,6k	2k muestras	31,93	2,13
`mnist_ddpm_v0.2`	38,6k	completo	**31,34**	2,14
`guided_mnist_ddpm_v0.1`	70,3k	completo	*provisional*	*provisional*
CIFAR-10				
`cifar_ddpm_v0.3`	45,7k	2k muestras	59,09	6,49
`cifar_ddpm_v0.3`	45,7k	completo	**41,24**	**6,94**
`guided_cifar_ddpm_v0.1`	52,7k	completo, $w = 2$	**36,04**	**8,28**

Tabla 2: Resultados con DDIM de 50 pasos (el DDPM original usa su propio muestreo). En negrita, los mejores valores medidos con el test completo. Las métricas de MNIST para el modelo guiado se consideran provisionales.

Muestreo	FID↓	IS↑
Clase nula (anterior)	42,62	6,72
Etiquetas, $w = 1$	38,63	7,33
Etiquetas, $w = 2$	**36,04**	8,28
Etiquetas, $w = 3$	38,99	8,50
Etiquetas, $w = 5$	43,98	8,64

Tabla 3: Barrido de CFG en CIFAR-10. El FID mínimo de los valores probados se obtiene con $w = 2$.

Muestreo	FID	IS
Clase nula (anterior)	34,34	2,16
Etiquetas, $w = 1$	37,35	2,32
Etiquetas, $w = 2$	53,03	1,55
Etiquetas, $w = 3$	44,99	1,94
Etiquetas, $w = 5$	33,31	2,30

Tabla 4: Barrido de CFG en MNIST. Resultados provisionales, pendientes de validar el evaluador.

tanto con como sin `-sin-etiquetas`, como cabe esperar al no depender de etiquetas.

Para MNIST, el mismo barrido produjo los siguientes valores con el evaluador LeNet:

Los resultados de MNIST son no monótonos y el IS es bajo para un conjunto de diez clases. Por ello no permiten concluir qué escala CFG es mejor. Antes de interpretarlos hay que medir el rendimiento del evaluador sobre imágenes reales, comprobar la clase predicha frente a la etiqueta solicitada, inspeccionar muestras y repetir algunas escalas con varias semillas. El FID de MNIST se calcula sobre un *embedding* LeNet de 128 dimensiones y no es comparable con el FID basado en Inception de CIFAR-10.

Métricas. En CIFAR-10, IS (10 particiones) y FID (raíz de covarianza con `eigh`) se calculan con Inception-v3 de torchvision. En MNIST se entrena un clasificador de tipo LeNet (tres convoluciones de 32, 64 y 128 canales con BatchNorm, *pooling* global y embedding de 128 dimensiones; AdamW, *early stopping* y selección en validación). Sus *logits* dan el IS y su embedding el FID, por lo que los valores de MNIST y CIFAR-10 no son comparables entre sí. Cada evaluación genera 10 000 imágenes (lote 512) y las compara con el test real.

Aplicación web. Un `ThreadingHTTPServer` de la biblioteca estándar (puerto 7860) expone `/api/models` y `/api/generate`. Detecta *checkpoints* con pesos EMA y permite elegir DDPM o DDIM, número de pasos, clase y escala de guiado. Devuelve la trayectoria inversa, que se recorre con un deslizador.

Reproducibilidad. Las dependencias están fijadas con `==` y la CI (GitHub Actions, Python 3.11) ejecuta `pytest`. Los tests cubren la ecuación directa, la reproducibilidad del muestreo, CFG, EMA, los *splits* y el servicio web. Los scripts de evaluación aceptan `-sin-etiquetas` para reproducir el muestreo anterior con clase nula y `-guidance-scale` para fijar w; en inferencia, `-label` permite seleccionar una clase concreta. La evaluación condicional usa etiquetas equilibradas por defecto. `tools/tiiac_check_demo.py` audita estructura, versiones, semillas y *smoke test*, y `AI_USAGE.md` registra el uso

de herramientas de IA.

4. Resultados

El modelo incondicional de CIFAR-10 obtiene FID 41,24 e IS 6,94 con DDIM de 50 pasos. Al evaluar el modelo guiado con etiquetas equilibradas, el mejor resultado del barrido realizado aparece en $w = 2$ (FID 36,04 e IS 8,28), mejor que el resultado anterior obtenido usando solo la clase nula. El IS aumenta en todo el intervalo probado, mientras que el FID empeora para escalas superiores a 2. Estos resultados son indicativos para este *checkpoint* y este protocolo, no una afirmación de optimalidad general. En MNIST, la variación irregular de FID e IS nos impide sacar conclusiones sobre la mejor asignación para el valor W.

5. Problemas durante el desarrollo

Durante el desarrollo nos encontramos con distintos problemas que nos dificultaron el desarrollo. Al principio tuvimos problemas con los gradientes a partir de las 20 epochs, la loss se disparaba a causa de que no normalizabamos los gradientes y resultaba en valores infinitos. Otro problema ha sido la desactualización de la configuración de los modelos originales sin guiado respecto a los cambios hechos para implementar el guiado con los nuevos modelos al usarlos en inferencia.

6. Trabajo pendiente

Queda evaluar los modelos usando el método DDPM en lugar de DDIM, que ha sido aplazado a causa de que es computacionalmente menos eficiente, e incluir las versiones iniciales de los modelos en las comparativas, 2. Para la versión 3.0 planeamos pulir el repositorio y, si hay tiempo, investigar las arquitecturas Diffusion Transformers (DiT).

Referencias

[1] J. Ho, A. Jain, P. Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020.

[2] J. Song, C. Meng, S. Ermon. *Denoising Diffusion Implicit Models*. ICLR, 2021.

[3] J. Ho, T. Salimans. *Classifier-Free Diffusion Guidance*. 2022.